

세션 4 · 장애대응 · 모니터링

지표 · 알람 · 장애 재현과 대응

대상 버전

Redis 8.10.x · MongoDB 8.3.x · Kafka 4.3.x

OS

Ubuntu 24.04 LTS

실습 환경

Azure

관측 도구

Prometheus · Grafana

01

관측 스택 구축

스택 생성

- 1 실습 PC 에서 `cd lab/terraform/04-operation`
- 2 `cp session.tfvars.example session.tfvars` 후 값 채우기
- 3 `terraform init`
- 4 `terraform plan -var-file=session.tfvars` 로 VM 10대 확인
- 5 `terraform apply -var-file=session.tfvars`
- 6 `terraform output` 으로 ops 노드 공인 IP 확인
- 7 그 공인 IP 로 SSH 접속

플레이북 실행

- 1 ops 노드에서 `cd ~/ansible`
- 2 `inventory/hosts.ini` 에 `[ops]` · `[redis]` · `[mongo]` · `[kafka]` 네 그룹이 있는 것 확인
- 3 `ansible -m ping all` 로 10대 응답 확인
- 4 `ansible-playbook play-operation.yml` 실행
- 5 마지막 요약에서 실패 노드가 없는 것 확인

Prometheus 수집 대상 확인

- **확인 절차:** ops 노드 공인 IP 의 **9090** 포트에서 **Status** 의 **Targets** 를 열고 대상 25개가 전부 **UP** 인 것 확인
- **node job 이 9대일 때:** ops 노드에 **node_exporter** 가 빠진 것

job	대상 수	수집 대상
node	10	서비스 노드 9대 + ops 노드. node_exporter · 9100
redis	3	redis_exporter · 9121
redis_sentinel	3	Sentinel 용 exporter · 26379
mongodb	3	mongodb_exporter · 9216
kafka	3	kafka_exporter · 9308
kafka_jmx	3	Broker JVM 의 java agent · 9404

Grafana 접속과 대시보드

- 1 ops 노드 공인 IP 의 **3000** 포트 접속
- 2 초기 계정으로 로그인
- 3 왼쪽 메뉴의 **Dashboards** 에서 **실습** 폴더 열기
- 4 대시보드 넷이 있는 것 확인

대시보드	담는 것
노드 지표	CPU · 메모리 · 디스크 · 네트워크. 열 대 전부
Redis 실습	메모리 · Persistence · 복제 · Sentinel
MongoDB 실습	복제 지연 · 멤버 상태 · WiredTiger 캐시
Kafka 실습	복제 · ISR · Consumer Lag · 처리량

세 서비스가 한 스택에 함께 올라오므로 대시보드도 한꺼번에 뜹니다.

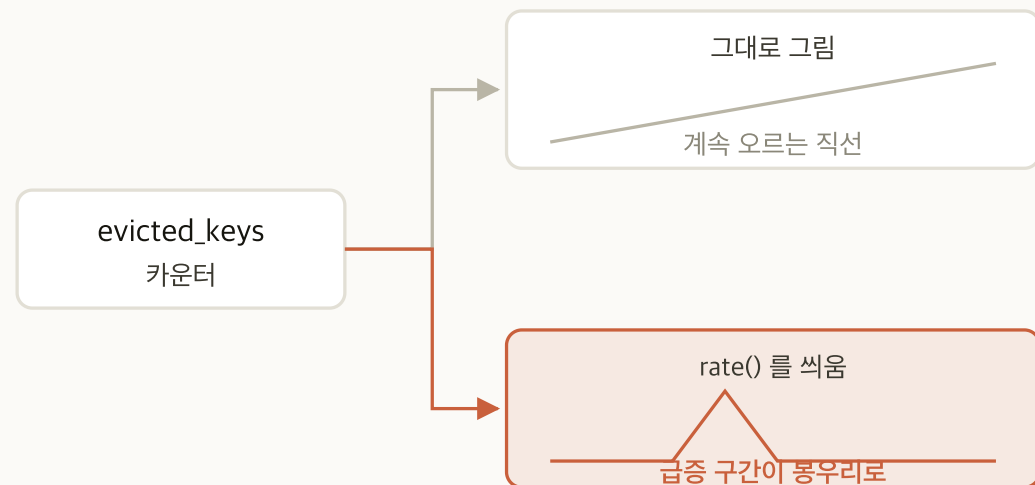
값이 그려지지 않으면 대시보드가 아니라 수집 문제입니다. 앞 장의 Targets 화면으로 돌아갑니다.

02

노드 지표와 로그

카운터와 게이지

- **게이지**: 지금 값. 오르내림. 메모리 사용량, 연결 수
- **카운터**: 누적 값. **단조 증가만**. 처리한 명령 수, 축출된 키 수
- 카운터를 그대로 그리면: 우상향 직선만 나옴. 아무것도 보이지 않음
- `rate()` 를 씌우면: 단위 시간당 증가량. **이것이 봐야 하는 값**



카운터에 `rate()` 를 씌우지 않으면 장애가 보이지 않습니다.

`evicted_keys` 가 계속 오르는 것은 당연합니다. 봐야 하는 것은 언제부터 빨리 오르기 시작했는가이고, 그것은 증가율에서만 보입니다.

CPU 와 부하

지표	확인 위치	의미	판단
<code>node_load1</code> ÷ vCPU 수	<code>node_exporter</code>	실행 대기 를 포함한 부하	1을 넘는 상태가 지속되면 CPU 포화
CPU <code>iowait</code> 비율	<code>node_cpu_seconds_total{mode="iowait"}</code>	IO 를 기다린 시간	높으면 병목이 CPU 가 아니라 디스크
CPU <code>steal</code> 비율	<code>node_cpu_seconds_total{mode="steal"}</code>	하이퍼바이저에 뺏긴 시간	클라우드에서만 보임. 온프레미스에서 는 0

`iowait` 이 높으면 CPU 를 늘려도 소용없습니다.

기다리고 있는 것이지 일하고 있는 것이 아닙니다. 이때는 디스크 지표로 넘어갑니다.

메모리와 스왑

지표	확인 위치	의미	판단
<code>node_memory_MemAvailable_bytes</code>	<code>node_exporter</code>	실제로 쓸 수 있는 메모리	<code>free</code> 가 아니라 이 값
스왑 입출력	<code>node_exporter</code>	스왑으로 나가고 들어온 양	0이 아니면 이미 낮음

- `free` 를 보면 안 되는 이유: page cache 가 「쓰고 있는 것」으로 잡힘. 필요하면 즉시 반납됨
- `available` 이 답하는 것: 지금 새 프로세스에 줄 수 있는 양
- Kafka 에서 특히: page cache 가 주역이라 `free` 는 거의 항상 작게 보임

Kafka 노드에서 `free` 가 작은 것은 정상입니다.

page cache 가 메모리를 채우고 있는 것이고, 그것이 Kafka 가 빠른 이유입니다. `available` 이 줄어들 때 걱정하면 됩니다.

디스크

지표	확인 위치	의미	판단
<code>node_filesystem_avail_bytes</code>	<code>node_exporter</code>	파일 시스템 여유	Kafka 는 이것이 1순위
디스크 사용률	<code>node_exporter</code>	디스크가 일한 시간의 비율	1에 가까우면 쉬지 않는 것
읽기·쓰기 처리량과 IOPS	<code>node_exporter</code> · 포털	실제 처리량	디스크 상한과 VM 상한 중 낮은 쪽과 대조

- **상한이 둘**: 디스크에 설정한 3,000 IOPS · 125 MB/s 와 VM 의 지속 처리량 상한
- **낮은 쪽이 실제 상한**: E2bds_v5 는 170 MB/s, D2ds_v5 는 **85 MB/s**

디스크가 차는 것은 예측할 수 있는 몇 안 되는 장애입니다.

갑자기 차지 않고 서서히 줄어듭니다. 남은 용량이 아니라 **줄어드는 속도로** 걸면 대응할 시간이 생깁니다.

서비스별 로그 경로와 검색어

대상	경로	검색 대상
Redis	/var/log/redis/redis-server.log	Background saving, MASTER <=> REPLICA sync, OOM
Sentinel	/var/log/redis/sentinel.log	+sdown, +odown, +switch-master
MongoDB	/var/log/mongodb/mongod.log	"c":"REPL", "c":"ELECTION", Slow query
Kafka	/opt/kafka/logs/server.log	ISR, Shrinking, NotEnoughReplicas
systemd	journalctl -u <유닛> --since	기동 실패, 재시작 반복
커널	dmesg -T	oom 으로 검색. 무엇이 죽었는가

증상만 보고 원인을 정하지 않습니다.

지표는 무엇이 이상한지까지 알려 주고, 왜 그런지는 로그가 알려 줍니다. 뒤의 장애 재현에서 이 경로를 그대로 씁니다.

노드 지표와 로그 점검

- 1 Grafana 노드 대시보드에서 열 대의 CPU · 메모리 · 디스크 여유 확인
- 2 `node_load1` 이 가장 높은 노드를 찾아 2 로 나눠 판단. 열 대 모두 `Standard_D2ds_v5` 이므로 vCPU 가 2
- 3 ops 노드에서 `ansible all -m shell -a "df -h"` 로 디스크 여유 확인
- 4 `ansible redis -b -m shell -a "tail -50 /var/log/redis/redis-server.log"` 로 정상일 때의 로그 모양을 봐둬. 장애 때 무엇이 평소와 다른지 이 기억으로 가름
- 5 `ansible all -b -m shell -a "dmesg -T | grep -i 'out of memory' || true"` 로 OOM 기록이 없는 것 확인

3번 명령의 예상 출력

```
redis-1 | CHANGED | rc=0 >>
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdc         64G  1.3G   63G   2% /var/lib/redis
```

세 서비스의 핵심 지표

보는 것	Redis	MongoDB	Kafka
포화	메모리 사용률 evicted_keys	WiredTiger 캐시 사용률 dirty 비율	데이터 디스크 사용량 JVM 힙 사용률
복제	connected_slaves master_link_up	복제 지연 멤버 상태	UnderReplicatedPartitions Partition 별 ISR 수
처리	초당 명령 수 명령별 평균 지연	검사한 키와 반환한 문서의 비 연결 수	BytesInPerSec Consumer Group 별 Lag
실패	rdb_last_bgsave_status aof_last_write_status	writeConflicts	OfflinePartitionsCount

서비스가 달라도 보는 갈래는 같습니다.

무엇이 차고 있는가, 복제가 붙어 있는가, 처리가 되고 있는가, 실패한 것이 있는가입니다. 서비스마다 이름만 다릅니다.

03

알람

알람의 세 등급

등급	뜻	적용 시점	예
페이지	사람을 깨움	지금 조치하지 않으면 나빠짐	leader 없는 Partition 발생, PRIMARY 부재
티켓	다음 근무일에 봄	지금은 버티지만 곧 문제	디스크 여유가 줄어드는 중
대시보드	알람 없이 봄	참고 값	초당 명령 수, 처리량

- **셋을 나누지 않으면:** 전부 페이지가 되고, 전부 무시됨
- **나누는 기준:** 지금 사람이 개입하지 않으면 나빠지는가

전부 페이지로 거는 것이 안전한 것이 아닙니다.

깨우는 알람이 많아지면 진짜 깨워야 할 때 반응이 늦어집니다. **페이지로 걸 것을 고를 때는 그 수를 세어 봅니다.**

for 절

- **하는 일**: 조건이 그 시간만큼 **계속** 참이어야 발동
- **없으면**: 순간적인 급등마다 알람. `scrape` 한 번 실패에도 울림
- **길게 잡으면**: 늦게 알게 됨. **짧게 잡으면**: 사람을 자주 깨우게 됨
- **디스크 규칙의 예**: 남은 양이 아니라 `predict_linear` 로 줄어드는 속도를 보고 `for: 30m`

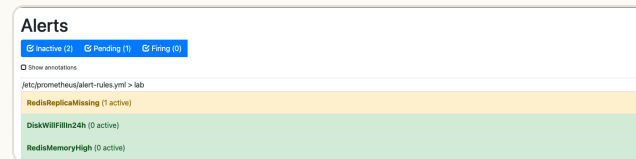


for 값 자체가 운영 판단입니다.

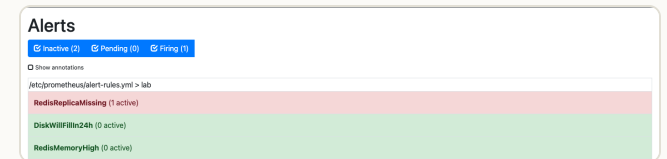
디스크 여유처럼 서서히 나빠지는 것은 길게, leader 상실처럼 즉시 영향이 나는 것은 짧게 잡습니다. 증상이 얼마나 빨리 나빠지는지가 그 값을 정합니다.

알람 확인 화면

- 1 ops 노드 공인 IP 의 9090 포트에서 Alerts 화면을 엽니다
- 2 배포된 규칙이 Inactive 로 올라온 것 확인
- 3 Grafana 실습 폴더의 노드 지표 대시보드에서 Edit · Add · Visualization
- 4 질의에 ALERTS{alertstate="firing"} 을 넣고 시각화를 Table 로 바꾼 뒤 저장
- 5 뒤 챕터의 장애 재현 중에 두 화면을 함께 열어 둡니다



조건은 참인데 시간이 안 찬 Pending



시간이 지나 Firing 으로 넘어간 상태

발동한 알람도 지표입니다.

Prometheus 가 ALERTS 라는 지표를 스스로 만들기 때문에 대시보드에서 그대로 질의할 수 있습니다. 지표를 보던 화면에서 알람을 함께 봅니다.

04

Redis 지표와 장애

Redis 지표의 출처

- **INFO memory** : 메모리 사용량, 단편화, 정책
- **INFO stats** : 히트율, 축출된 키, fork 시간, 재동기화 횟수
- **INFO replication** : master 연결 상태, replica 수, 복제 offset
- **INFO clients** : 연결 수, 블로킹 대기
- **SLOWLOG GET** : 느린 명령의 원문. **지표가 아니라 목록**

SLOWLOG 는 대시보드에 나오지 않습니다.

느린 명령이 무엇이었는지는 지표에 나오지 않고 그 자리에서 조회해야 합니다. 느린 명령 하나가 전체를 막을 수 있으므로 장애를 만나면 이 명령을 함께칩니다.

메모리 지표

INFO 필드	Prometheus 지표	의미	판단
<code>used_memory</code> ÷ <code>maxmemory</code>	<code>redis_memory_used_bytes</code> ÷ <code>redis_memory_max_bytes</code>	메모리 사용률	80% 이상이면 eviction 임박
<code>maxmemory_policy</code>	<code>redis_instance_info</code> 의 label	한계 도달 시 동작	<code>noeviction</code> 이면 쓰기가 거부됨
<code>evicted_keys</code> 증가율	<code>redis_evicted_keys_total</code>	축출된 키 수	0이 아니면 캐시가 모자람
<code>expired_keys</code> 증가율	<code>redis_expired_keys_total</code>	TTL 로 사라진 키 수	<code>evicted_keys</code> 와 구분. 이쪽은 정상
<code>mem_fragmentation_ratio</code>	<code>redis_mem_fragmentation_ratio</code>	RSS ÷ <code>used_memory</code>	1.5 이상은 단편화, 1 미만은 스왑

`evicted_keys` 와 `expired_keys` 를 섞으면 안 됩니다.

TTL 로 사라지는 것은 설계대로 도는 것이고, 축출되는 것은 메모리가 모자란 것입니다. 둘을 합쳐 보면 정상 동작을 장애로 읽습니다.

복제 지표

INFO 필드	Prometheus 지표	의미	판단
<code>connected_slaves</code>	<code>redis_connected_slaves</code>	master 가 본 replica 수	구성값보다 적으면 이탈
<code>master_link_status</code>	<code>redis_master_link_up</code> · <code>redis_up</code>	연결 상태와 Redis 응답	0이면 끊겼거나 죽은 것
복제 offset 의 차	<code>redis_connected_slave_offset_bytes</code>	각 replica 의 복제 offset	차가 벌어지면 못 따라잡는 것
<code>role</code>	<code>redis_instance_info</code> 의 label	master 인가 replica 인가	알람을 줄힐 때
<code>sync_full</code> · <code>sync_partial_ok</code>	<code>redis_replica_resyncs_full</code> · <code>_partial_resync_accepted</code>	전체·부분 재동기화 횟수	full 이 반복되면 부족
<code>master_sync_in_progress</code>	<code>redis_master_sync_in_progress</code>	재동기화 중인가	1이면 기다림

PromQL 기초

Prometheus 의 **Graph** 화면에서 실행합니다

```
# label 로 고르기
node_memory_MemAvailable_bytes{service="redis"}

# 카운터의 증가율
rate(redis_evicted_keys_total[5m])

# 노드별 값을 하나로 모으기
avg by (instance) (
    rate(node_cpu_seconds_total{mode="idle"}[5m]))

# 두 지표의 비
redis_memory_used_bytes / redis_memory_max_bytes
```

- **{}** : label 로 대상 좁히기. **service** label이 여기서 쓰임
- **[5m]** : 최근 5분 구간. **rate()** 와 함께 씀
- **by (instance)** : 노드별로 나눠 집계

알람 규칙 두 건 작성

/etc/prometheus/alert-rules.yml 끝에 붙입니다

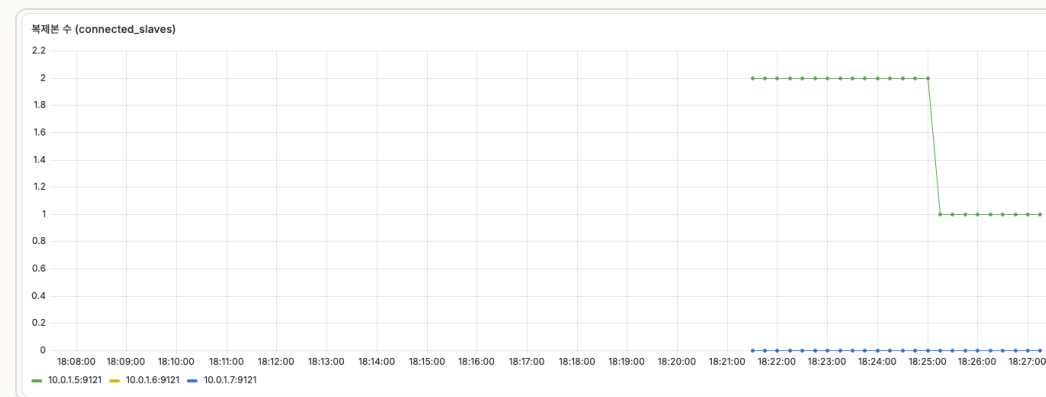
```
- alert: RedisMemoryHigh
  expr: redis_memory_used_bytes
    / redis_memory_max_bytes > 0.9
  for: 5m
  labels: {severity: ticket}
- alert: RedisReplicaMissing
  expr: redis_connected_slaves < 2
    and on(instance)
    redis_instance_info{role="master"}
  for: 1m
  labels: {severity: page}
```

- 1 ops 노드에서 `sudo vi /etc/prometheus/alert-rules.yml`
- 2 위 두 규칙을 파일 끝에 붙여 넣음
- 3 `sudo systemctl reload prometheus`
- 4 Alerts 화면에서 세 규칙 확인

서비스	데이터 디스크 마운트 지점
Redis	/var/lib/redis · RDB·AOF
MongoDB	/var/lib/mongodb · 데이터 파일·oplog
Kafka	/var/lib/kafka · segment 파일

장애: replica 하나 이탈

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-r01.sh` 실행
- 3 Grafana 의 「replica 수」 패널에서 값이 2에서 1로 떨어지는 것 확인
- 4 Alerts 화면에서 복제 규칙이 Pending 을 거쳐 Firing 이 되는 것 확인
- 5 이 시각과 두 화면을 적어 둬. 감별의 출발점



「replica 수」 패널. 값이 2에서 1로

알람이 울리는 것까지가 증상입니다.

앞서 건 규칙이 여기서 실제로 발동합니다.

replica 이탈의 원인 후보와 감별

원인	판별 근거
replica 프로세스 정지	<code>redis_up</code> 이 0. 두 exporter 의 target 은 <code>UP</code> 유지
노드 정지	두 exporter 의 target 이 함께 <code>DOWN</code>
네트워크 차단	노드는 살아 있으나 복제만 끊김
전체 재동기화 진행 중	<code>master_sync_in_progress</code> 가 1
<code>repl-backlog</code> 부족	<code>sync_full</code> 이 반복해서 오름

- 1 ops 노드에서 `redis-cli -h <master 사실 IP> INFO replication`
- 2 `connected_slaves` 와 replica 줄에서 빠진 노드를 찾음
- 3 Prometheus 에서 `redis_up` 과 `up{job="redis"}` · `up{job="node"}` 를 함께 조회
- 4 남은 replica 에서 `master_link_status` 와 `master_sync_in_progress` 확인
- 5 `ansible redis -m shell -a "tail -50 /var/log/redis/sentinel.log"` 로 `+sdown` · `+odown` 확인

첫 갈래는 노드인가 서비스인가입니다.

`redis_up` 이 0인데 두 target 이 `UP` 이면 프로세스 정지이고, 두 target 이 함께 `DOWN` 이면 노드 정지입니다.

감별에서 보는 출력

```
# master 에서
role:master
connected_slaves:1
slave0:ip=10.0.1.7,port=6379,
    state=online,offset=125843,lag=1

# 빠진 replica 에서
Could not connect to Redis at
10.0.1.9:6379: Connection refused

# 남아 있는 replica 에서
master_link_status:up
master_sync_in_progress:0
```

- **connected_slaves:1** : 구성값 2보다 적음. 이탈 확인
- **Connection refused** : 프로세스가 죽은 것. 이 거부가 곧 감별 근거
- 남은 replica 의 **up** : 끊긴 것이 한 대뿐임을 확인
- **master_link_status:down** : 프로세스는 살아 있고 경로만 끊긴 갈래에서 나옴. 지금 재현한 정지에서는 볼 수 없음

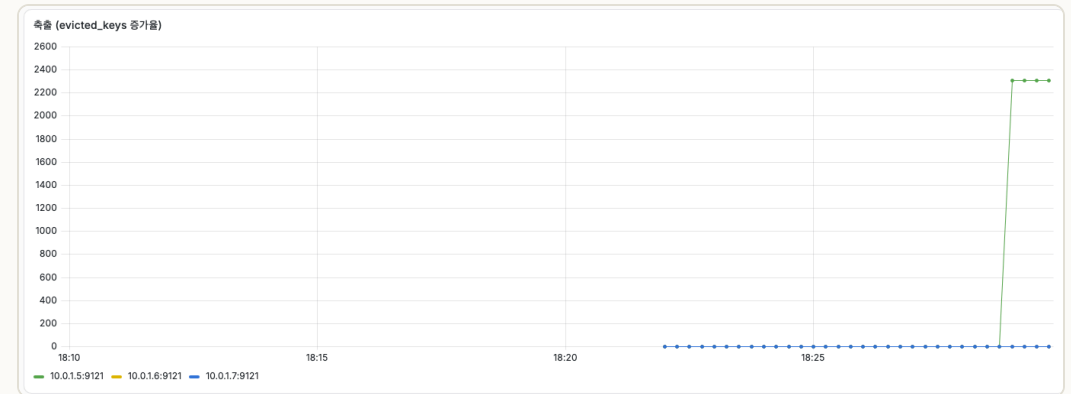
replica 이탈의 완화와 복구

원인	완화	복구
프로세스 정지	<code>./rb-r01.sh --restore</code>	<code>REPLICA0F</code> 유지 확인
노드 정지	노드 복구	자동 재접속 확인
네트워크 차단	경로 복구	자동 재접속 확인
전체 재동기화 반복	재접속을 잠시 억제	<code>repl-backlog-size</code> 상향

- **복구 검증:** master 에서 `connected_slaves` 가 2
- 각 replica 에서 `master_link_status` 가 `up`
- `SENTINEL replicas mymaster` 가 둘을 반환
- Grafana 그래프가 원래 값으로 돌아오고 알람이 `Inactive`

장애: 히트율 하락과 쓰기 실패

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-r02.sh 1` 실행. 상한을 낮추고 적재한 뒤 같은 키 공간을 읽음
- 3 Grafana 의 「축출」, 「히트율」 패널에서 값이 바뀌는 것 확인
- 4 Alerts 화면에서 Redis 메모리 규칙이 Firing 이 되는 것 확인
- 5 VM 메모리 그래프에 변화가 없는 것도 확인
- 6 `./rb-r02.sh 2` 실행. 정책을 `noeviction` 으로 바꾸고 상한을 지금 사용량 아래로 내려 쓰기 거부를 유발
- 7 `redis-cli -h <노드 사설 IP> set probe:oom 1` 로 쓰기 거부 확인



「축출」 패널. evicted_keys 상승

노드 지표가 정상인데 서비스가 나빠지고 있습니다.

노드 지표를 먼저 보고 「노드가 아니다」로 갈린 경우입니다.

감별 절차와 정책에 따른 증상 차이

- 1 ops 노드에서 `redis-cli -h <노드 사설 IP> INFO memory`
- 2 `used_memory` 와 `maxmemory` 의 비 확인
- 3 `maxmemory_policy` 확인. 여기서 eviction 과 쓰기 거부가 갈림
- 4 `INFO stats` 에서 `evicted_keys` 증가율과 히트율 확인
- 5 `mem_fragmentation_ratio` 로 스왑 여부 확인
- 6 노드 메모리 여유와 커널 로그 확인. 서비스 문제인지 노드 문제인지 가름

정책	한계에 닿았을 때
<code>allkeys-lru</code>	오래된 키를 축출함. 쓰기는 되지만 히트율이 떨어짐
<code>noeviction</code>	쓰기를 거부함. <code>OOM command not allowed</code>

같은 메모리 부족인데 증상이 정반대로 보입니다.

한쪽은 「느려졌다」이고 다른 쪽은 「오류가 난다」입니다. 정책을 확인하지 않으면 같은 원인을 두 장애로 읽습니다.

메모리 압박의 완화와 복구

원인	완화	복구
eviction 폭주	정책을 상황에 맞게 조정	<code>maxmemory</code> 재산정, TTL 설계 점검
쓰기 거부	정책을 전환해 쓰기를 우선 수용	용량 재산정
스왑	부하 감소	노드 메모리 재산정
fork 시 급증	저장 주기 조정	<code>maxmemory</code> 를 fork 여유까지 낮춤

- **복구 검증:** `evicted_keys` 증가가 멈추고 히트율이 회복
- `OOM command not allowed` 가 더 나오지 않음
- `./rb-r02.sh --status` 로 `maxmemory` 와 정책이 원래 값
- 알람이 `Inactive` 로 돌아옴

먼저 `./rb-r02.sh --restore` 를 실행합니다.

주입 스크립트는 스스로 되돌리지 않습니다. 이 명령을 실행하지 않으면 `maxmemory` 가 낮은 값으로 남아 이후 실습에서 쓰기가 계속 거부됩니다.

05

MongoDB 지표와 장애

MongoDB 지표의 출처

- `rs.status()` : 멤버 상태, 각 멤버의 마지막 반영 시각
- `db.serverStatus()` : 캐시, 대기 큐, 연결, 처리량, 지연 분포
- `db.getReplicationInfo()` : oplog 가 담고 있는 시간 범위
- `db.currentOp()` : 지금 도는 작업. **지표가 아니라 목록**
- `mongod.log` : 느린 질의 원문, 선출 기록

`db.currentOp()` 는 그 순간에만 볼 수 있습니다.

오래 도는 질의가 무엇인지는 지표에 남지 않습니다. 장애 중에 조회해야 하고, 끝난 뒤에는 로그의 느린 질의 기록에서만 찾을 수 있습니다.

복제 지표

지표	Prometheus 지표	의미	판단
멤버별 <code>optimeDate</code> 차	<code>mongodb_members_optimeDate</code>	복제 지연	oplog 윈도우에 가까워지면 위험
oplog 윈도우	없음. <code>db.getReplicationInfo()</code> 로만	oplog 가 담은 시간 범위	복구 여유의 직접 지표
<code>health</code>	<code>mongodb_members_health</code>	멤버에 닿는가	0이면 그 멤버가 끊긴 것
<code>stateStr</code>	<code>rs.status()</code> 로만	멤버 역할 문자열	PRIMARY 1 · SECONDARY 2 가 정상
election 발생 횟수	없음. <code>rs.status()</code> 와 로그	선출이 몇 번 일어났는가	반복되면 네트워크나 시계 문제

지연 자체보다 oplog 윈도우와의 거리가 중요합니다.

지연 30초는 윈도우가 24시간이면 여유롭고 윈도우가 1분이면 이미 위험합니다. 두 값을 함께 봐야 판단이 됩니다.

WiredTiger 캐시 지표

지표	확인 위치	의미	판단
cache used 비율	wiredTiger.cache · mongodb_ss_wt_cache_ 계열	캐시 사용률	80%가 압박 신호
dirty 비율	같은 곳	아직 디스크에 안 내려간 비율	20%가 압박 신호
요청 스레드가 수행한 eviction	같은 곳	요청 스레드가 대신 수행	0이 아니면 이미 심각
캐시로 읽어 들인 바이트 증가율	같은 곳	디스크에서 캐시로 올라온 양	크면 working set 이 캐시를 넘은 것

요청 스레드 eviction 이 가장 확실한 신호입니다.

평소에는 전용 스레드가 eviction 을 처리합니다. 전용 스레드가 따라가지 못해 **질의**를 처리하던 스레드가 **대신하기 시작하면** 그 질의는 그만큼 늦어집니다. 사용자가 체감하는 순간입니다.

질의 효율 지표

지표	확인 위치	의미	판단
scanned 대 returned 비	<code>db.serverStatus()</code> 의 <code>metrics</code>	검사한 문서 대비 반환	크면 인덱스가 없거나 잘못됨
<code>scanAndOrder</code>	<code>metrics.operation</code>	인덱스로 정렬하지 못한 횟수	정렬용 인덱스가 없는 것
slow query 기록	<code>mongod.log</code> 와 profiler	느린 질의 원문	기본 임계값은 100ms
<code>opLatencies</code>	<code>db.serverStatus()</code>	읽기·쓰기·명령 지연 분포	평균이 아니라 분포
<code>opcounters</code>	<code>db.serverStatus()</code>	연산 종류별 처리량	패턴 변화 감지

scanned 대 returned 비가 1에 가까우면 인덱스가 잘 맞는 것입니다.

1000개를 훑어 1개를 돌려주고 있으면 그 질의는 인덱스를 못 쓰고 있습니다. 포털의 디스크 IOPS 그래프가 답하지 못하는 '왜'가 여기 있습니다.

MongoDB 알람 규칙 작성

/etc/prometheus/alert-rules.yml 끝에 붙입니다

```
- alert: MongoReplicationLagHigh
  expr: (max by (rs_nm) (
    mongodb_members_optimeDate{rs_state="1",
    member_state="PRIMARY"} > 0) - on(rs_nm)
    group_right() max by (rs_nm, member_idx)
    (mongodb_members_optimeDate{rs_state="1"}
    > 0)) / 1000 > 60
  for: 2m
  labels: {severity: ticket}
- alert: MongoMemberDown
  expr: mongodb_members_health == 0
  for: 1m
  labels: {severity: page}
```

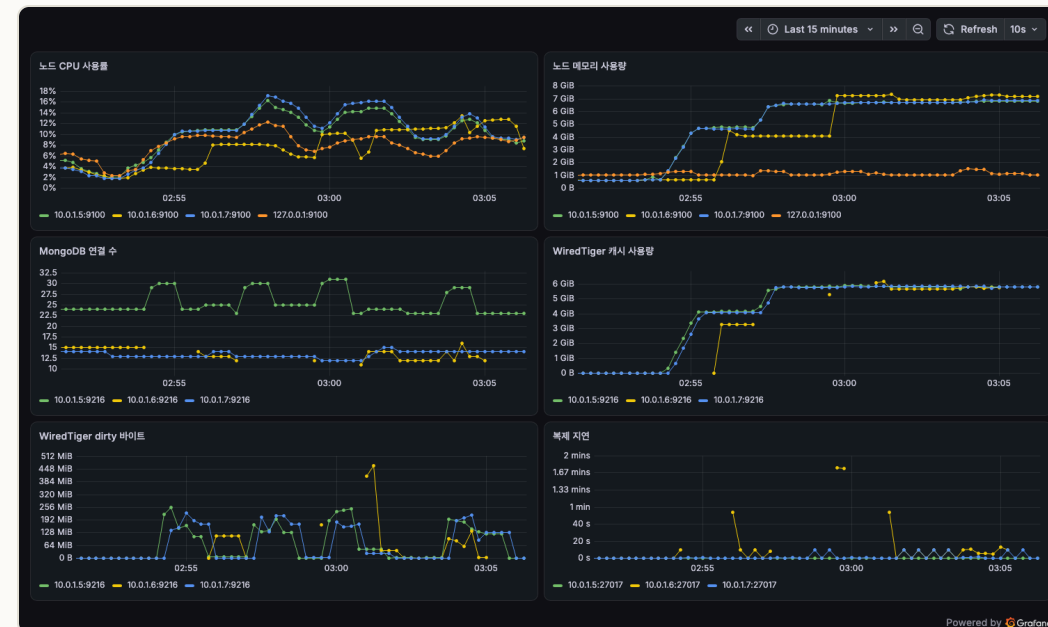
- 1 ops 노드에서 `sudo vi /etc/prometheus/alert-rules.yml`
- 2 위 두 규칙을 파일 끝에 붙여 넣음
- 3 `reload prometheus` 후 Alerts 확인

지연 규칙만 걸면 가장 나쁜 상태에서 아무것도 울리지 않습니다.

멤버가 완전히 끊기면 `optimeDate` 가 `1970-01-01` 이 되고 지연 식의 `> 0` 필터가 그 멤버를 제외합니다. 그래서 `health` 를 함께 겁니다.

장애: 복제 지연 증가

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-m01.sh` 실행
- 3 Grafana 의 「복제 지연」 패널에서 한 멤버의 지연이 우상향하는 것 확인
- 4 Alerts 화면에서 복제 지연 규칙이 `Firing` 이 되는 것 확인
- 5 이 시각과 두 화면을 적어 둬. 감별의 출발점



「복제 지연」 패널. 한 멤버가 우상향

지연이 「크다」가 아니라 「늘고 있다」가 문제입니다.

일정하게 유지되는 지연은 따라가고 있다는 뜻입니다. 벌어지면 못 따라가는 것입니다.

복제 지연의 원인 후보와 감별

원인	판별 근거
SECONDARY 이탈	<code>rs.status()</code> 에서 해당 멤버가 unreachable
oplog 를 못 따라감	CPU·디스크 IO 상승, 또는 반대로 CPU 가 낮는데 지연만 벌어짐
대량 쓰기 유입	<code>opcounters</code> 급증
인덱스 빌드 중	<code>db.currentOp()</code> 에 빌드 작업
네트워크 대역 부족	노드 지표의 수신량이 상한에 붙음

- 1 ops 노드에서 `mongosh --host <PRIMARY 사설 IP>`
- 2 `rs.status()` 로 멤버 상태와 `optimeDate` 확인
- 3 `rs.printSecondaryReplicationInfo()` 로 지연 크기 확인
- 4 `db.getReplicationInfo()` 로 oplog 윈도우 확인
- 5 지연 노드의 CPU 와 디스크 확인. 낮으면 부하가 아니라 프로세스 쪽
- 6 `opcounters` 로 PRIMARY 의 쓰기 유입량 확인

지연이 oplog 윈도우의 몇 퍼센트인지가 복구 수단을 가릅니다.

재현 중 실측으로 지연 93초에 윈도우 184초였습니다. 50.5% 로 이미 위험한 구간이고, 이 사례처럼 절반을 넘기면 초기 동기화 말고는 수단이 없습니다.

복제 지연의 완화와 복구

원인	완화	복구
이탈	노드 복구	자동 따라잡기 확인
oplog 를 못 따라감	그 노드로 가는 읽기 차단	부하 원인 제거
대량 쓰기	유입 속도 조절	용량 재산정
인덱스 빌드	완료를 기다림	부하가 적은 때로 옮김
네트워크 대역 부족	다른 트래픽 억제	경로 점검

- **복구 검증:** 모든 멤버의 `health` 가 1
- `stateStr` 이 PRIMARY 하나와 SECONDARY 둘
- **복제 지연이 0초로 수렴.** 줄기 시작한 것만으로는 부족함
- `./rb-m01.sh --status` 로 확인하고 알람이 `Inactive`

먼저 `./rb-m01.sh --restore` 를 실행합니다.

주입 스크립트는 스스로 되돌리지 않습니다. oplog 윈도우를 넘긴 뒤에는 완화 수단이 없고 초기 동기화가 끝날 때까지 그 멤버가 정족수에 들어가지 않습니다.

06

Kafka 지표와 장애

Kafka 지표의 출처

출처	내는 지표	노드 정지 시 영향
JMX exporter	Broker 내부 MBean. 복제·controller·요청 처리	Broker 와 함께 끊김
kafka_exporter	Topic · Partition · Consumer Group Lag	그 노드 몫만 끊김
node_exporter	디스크 여유, CPU, 메모리, 파일 디스크립터	노드가 죽어야 끊김
CLI	배치와 상태의 상세	명령을 칠 때만

어디서 나오는지 곧 「그 노드가 죽으면 무엇이 보이지 않는가」입니다.

Broker 가 죽으면 그 Broker 의 JMX 지표는 사라지지만 `node_exporter` 는 남습니다. 노드인지 서비스인지를 가르는 판단이 Kafka 에서는 이 모양입니다.

복제 지표

지표	확인 위치	의미	판단
UnderReplicatedPartitions	JMX	ISR 이 RF 에 못 미치는 Partition 수	0이 아니면 즉시 확인
UnderMinIsrPartitionCount	JMX	ISR 이 min.insync.replicas 미만인 수	0이 아니면 acks=all produce 가 거부 됨
OfflinePartitionsCount	JMX	leader 가 없는 Partition	0이 아니면 완전히 멈춘 것
ActiveControllerCount	JMX	활성 controller 수. Broker 별 값	합이 정확히 1
ISR 축소·확장 횟수	JMX	ISR 축소와 확장의 빈도	반복되면 네트워크나 디스크

UnderReplicated 와 UnderMinIsr 는 다릅니다.

앞은 서비스가 돌아가고, 뒤는 acks=all 쓰기가 거부되고 있는 상태입니다.

처리와 디스크 지표

지표	확인 위치	의미	판단
RequestHandlerAvgIdlePercent	JMX	요청 처리 스레드 유휴율	낮으면 Broker 포화
BytesInPerSec · BytesOutPerSec kafka_server_bytes_in_total	JMX	처리량. 카운터라 rate() 가 필요함	Lag 감별에서 Producer 급증을 배제
데이터 디스크 여유와 flush 지연	node_exporter · JMX	log dir 이 있는 파일 시스템	Kafka 장애 원인 1순위
열린 파일 디스크립터	node_exporter	열린 파일 수	segment 마다 파일을 여는 구조
JVM 힙 사용률	jvm_memory_used_bytes{area="heap"}	Broker JVM 의 힙	힙을 키우면 page cache 가 줄어듦
Consumer Group 별 Lag	kafka_exporter	LEO 에서 커밋된 Offset 을 뺀 값	챕터 11 실습의 주역

Topic 과 Consumer Group 생성

- 1 `kafka-topics.sh --bootstrap-server <Broker 사설 IP>:9092 --create --topic lab.events --partitions 3 --replication-factor 3 --config min.insync.replicas=2`
- 2 `--describe --topic lab.events` 로 **Partition 3개와 ISR 3개** 확인
- 3 `kafka-console-producer.sh` 로 메시지 몇 건 입력 후 종료
- 4 `kafka-console-consumer.sh --group ops-cg --from-beginning` 실행 후 종료
- 5 `kafka-consumer-groups.sh --describe --group ops-cg` 로 **LAG 열의 정상시 값** 확인

`--replication-factor 3` 을 빼면 Broker 한 대가 멈출 때 그 Partition 이 통째로 멈춥니다.

Broker 가 3대이므로 3으로 둡니다. `min.insync.replicas=2` 는 셋 중 둘이 살아 있어야 `acks=all` 쓰기를 받는다는 뜻입니다.

여기서 본 **LAG** 값이 다음 장 임계값의 근거입니다.

Lag 은 정상 운영에서도 0이 아닙니다. 지금 값을 적어 두어야 임계 값을 정할 수 있습니다.

Kafka 알람 규칙 작성

/etc/prometheus/alert-rules.yml 끝에 붙입니다

```
- alert: ConsumerLagHigh
  expr: sum by (consumergroup) (max by
    (consumergroup, topic)
    (kafka_consumergroup_lag_sum)) > 10000
  for: 3m
  labels: {severity: ticket}
- alert: UnderReplicatedPartitions
  expr:
    kafka_server_under_replicated_partitions
    > 0
  for: 2m
  labels: {severity: page}
```

- 1 ops 노드에서 `sudo vi /etc/prometheus/alert-rules.yml`
- 2 위 두 규칙을 파일 끝에 붙여 넣음
- 3 `sudo systemctl reload prometheus` 후 Alerts 화면 확인

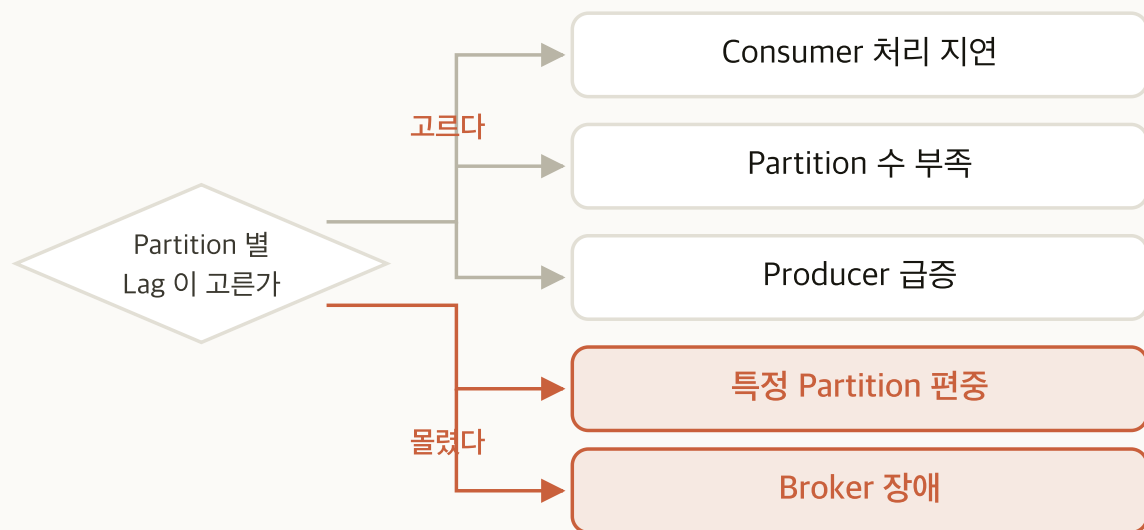
Lag 은 정상 운영에서도 0이 아닙니다.

앞서 켜 평상시 LAG 값 위에서 임계를 잡습니다. max 로 한 번 거르는 것은 Broker 세 대의 exporter 가 같은 클러스터를 각각 보고하기 때문입니다.

Consumer Lag 의 원인 여섯

원인	판별 근거	대응
Consumer 처리 지연	Consumer CPU 상승, Lag 이 고르게 증가	처리 로직 개선, Consumer 증설
Partition 수 부족	Consumer 를 늘려도 Lag 이 줄지 않고 일부가 유휴	Partition 증설
Rebalance 반복	Consumer 로그의 Rebalance 반복	<code>max.poll.interval.ms</code> 조정, 배치 축소
Broker 장애	<code>UnderReplicatedPartitions</code> 증가	Broker 복구
Producer 급증	<code>rate()</code> 를 씌운 <code>BytesInPerSec</code> 급증. Lag 은 결과일 뿐	용량 증설 또는 유량 제어
특정 Partition 편중	Lag 이 한쪽에만 쌓임	키 분포 확인, Partitioner 점검

Lag 분포로 가르는 첫 갈래



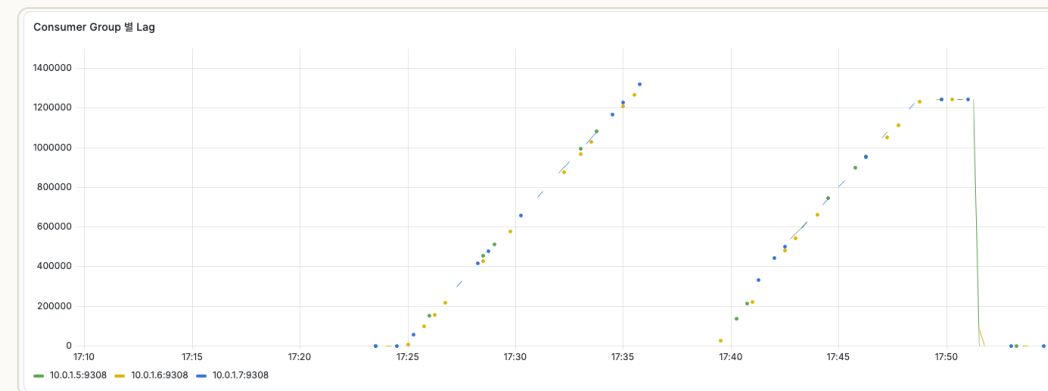
- 한 번의 조회로 여섯이 넷으로: `--describe` 한 번이면 분포가 나옴
- 고르면: Consumer 쪽이거나 유입 쪽
- 몰렸으면: 키 분포거나 특정 Broker
- Rebalance 반복은 별도: 분포는 고르고 로그에서만 갈림

분포를 먼저 보면 불필요한 조치가 줄어듭니다.

Consumer 증설에는 비용과 시간이 듭니다. 명령 한 번으로 후보가 여섯에서 넷으로 줄면 그다음 조치가 정확해집니다.

장애: Consumer Lag 증가

- 1 ops 노드에서 `cd ~/lab/scenarios`
- 2 `./rb-k01.sh` 실행
- 3 Grafana 의 「Consumer Group 별 Lag」 패널에서 Lag 이 우상향하는 것 확인
- 4 Alerts 화면에서 Lag 규칙이 Firing 이 되는 것 확인
- 5 적어 둔 평상시 LAG 값과 대조



「Consumer Group 별 Lag」 패널. 우상향

Lag 이 늘었다는 것만으로는 원인을 모릅니다.

여섯 갈래 중 무엇인지 앞 세 장의 감별 순서로 좁혀 갑니다.

Lag 의 감별 절차

- 1 ops 노드에서 `kafka-consumer-groups.sh --bootstrap-server <IP>:9092 --describe --group <그룹>`
- 2 Partition 별 Lag 분포를 봄. 고른 분포인지 편중인지로 1차 분류
- 3 `rate(kafka_server_bytes_in_total[5m])` 그래프로 Producer 급증 여부 배제
- 4 분포가 몰렸을 때만: `UnderReplicatedPartitions` 로 Broker 장애 여부 배제
- 5 `CURRENT-OFFSET` 이 전혀 움직이지 않는 Partition 이 있는지 확인
- 6 Consumer 로그에서 Rebalance 발생 여부 확인
- 7 남은 후보가 Partition 수 부족이면 Consumer 를 하나 늘려 보고 Lag 변화 관찰

Lag 분포 확인

Consumer group 'ops-cg' has no active members. 가 먼저 뜬 뒤

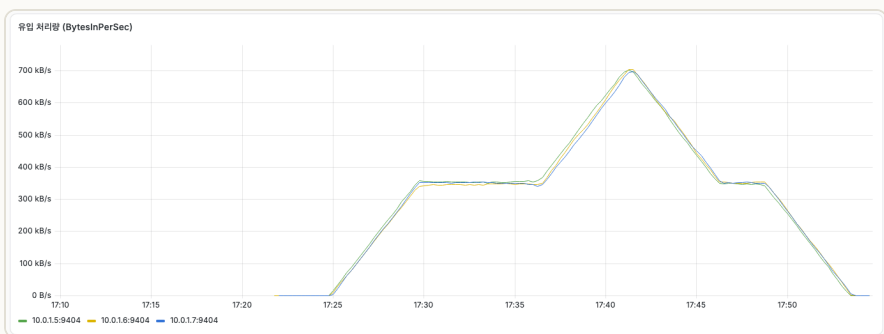
GROUP	TOPIC	PARTITION	CURRENT	LOG-END	LAG
ops-cg	lab.events	1	230299	242009	11710
ops-cg	lab.events	0	-	233869	-
ops-cg	lab.events	2	-	235875	-

- **CURRENT-OFFSET** 이 있어도 활성 멤버가 없을 수 있음: Consumer 가 전부 그룹에서 빠진 상태
- **CURRENT-OFFSET** 이 - 인 Partition: 맡은 Consumer 가 커밋하기 전에 재할당됨

「-」 표시를 Lag 숫자보다 먼저 봅니다.

커밋된 Offset 이 없는 Partition 은 Lag 을 계산할 수 없습니다. 숫자를 읽기 전에 몇 개가 - 인지부터 봅니다.

Producer 급증 배제



rate() 를 씌운 BytesInPerSec

- 유입량이 그대로: Producer 급증 배제
- 남은 후보 셋: Consumer 처리 지연, Partition 수 부족, Rebalance 반복
- 다음 단계: Consumer 로그 확인

Producer 급증부터 배제해야 원인을 잘못 지목하지 않습니다.

유입이 두 배가 된 상황이라면 Consumer 는 정상이고 용량 문제입니다. Consumer 를 아무리 손봐도 해결되지 않습니다.

Consumer 로그에서 확정

```
ops-1: grep -i rebalance /tmp/rkm-consumer-*.log

> grep -hoIE "rebalanc[a-z]*|assign[a-z]*" /tmp/rkm-consumer-*.log | sort | uniq -c
40 assignment
12 rebalance

# 90초 동안 Rebalance 가 12회, 재할당이 40회 일어났다.
# max.poll.interval.ms 를 넘겨 그룹에서 제외되고, 다시 들어오면서 또 일어난다.
#
# 이 값은 지표로 나오지 않는다. Consumer JVM 에 별도 agent 를 붙이지 않기
# 때문이다. 여섯 갈래 중 로그를 봐야만 갈리는 유일한 갈래다.
```

Rebalance 가 반복되는 구간

- **어디서:** /tmp/rkm-consumer-1.log 부터 -3.log . 주입 스크립트가 Consumer 출력을 남기는 자리
- **찾을 것:** 그룹 재가입과 재할당이 짧은 간격으로 반복되는 기록
- **함께 보이는 것:** poll 간격 초과로 그룹에서 제외됐다는 기록
- **확정: Rebalance 반복**
- **지표로 갈리지 않는 이유:** Consumer JVM 에 별도 agent 를 붙이지 않아 이 값은 Prometheus 에 들어오지 않음

여섯 갈래 중 지표만으로 끝나지 않는 유일한 갈래입니다.

Lag 분포는 고르고 유입량도 정상이라 지표 화면에서는 「Consumer 가 느리다」까지만 보입니다. 왜 느린지는 로그에 있습니다.

Lag 의 완화와 복구

원인	완화	복구
Consumer 처리 지연	Consumer 증설	처리 로직 개선
Partition 수 부족	즉시 완화 수단 없음	Partition 증설. 되돌릴 수 없음
Rebalance 반복	가져오는 건수를 줄임	<code>max.poll.interval.ms</code> 조정. static membership 적용
Broker 장애	남은 Broker 로 버팀	Broker 복구
Producer 급증	유량 제어	용량 증설

복구 검증은 Lag 이 다시 내려오는 것까지입니다.

Rebalance 기록이 끊기고, 모든 Partition 의 `CURRENT-OFFSET` 이 함께 전진하고, Lag 규칙이 `Inactive` 로 돌아옵니다.

주입을 멈추는 것과 Lag 이 사라지는 것은 다릅니다.

먼저 `./rb-k01.sh --restore` 를 실행합니다. 이미 쌓인 것은 정상 Consumer 가 읽어 내야 사라집니다.

07

과정 마무리

메모리 압박 3종 대비

	Redis	MongoDB	Kafka
부족 시 증상	eviction 또는 쓰기 거부	디스크 IO 로 강등	page cache 축출
성능 저하 양상	계단식	계단식	점진적
먼저 보는 지표	메모리 사용률, <code>evicted_keys</code>	cache used·dirty 비율	힙 사용률, 디스크 읽기량
확정하는 지표	히트율 하락	요청 스레드 eviction	page cache 대비 디스크 읽기
흔한 오설정	<code>maxmemory</code> 미설정	캐시를 넘는 working set	힙을 크게 잡음
대응	정책 조정, 증설	인덱스 정리, 증설	힙을 줄임
잘못된 대응	<code>maxmemory</code> 를 물리까지 올림	캐시를 물리까지 올림	힙을 늘림

클라우드 비의존 범위

단계	Azure	온프레미스나 다른 클라우드에서
인프라 생성	Terraform	바뀜. 제공자에 맞게
구성	Ansible	그대로
관측	Prometheus · Grafana	그대로
인프라 지표	포털 · <code>node_exporter</code>	<code>node_exporter</code> 만으로
장애 대응	각 서비스의 명령과 로그	그대로
알람 규칙	Prometheus 규칙 파일	그대로

클라우드 의존은 인프라 생성 단계까지였습니다.

그 이후의 구성 · 운영 · 모니터링 · 장애 대응은 어디서든 그대로 통합니다.

리소스 정리

- 1 실습 PC 의 `lab/terraform/04-operation` 에서 `terraform destroy -var-file=session.tfvars`
- 2 포털에서 리소스 그룹이 사라진 것 확인

destroy 는 VM 열 대와 디스크·네트워크를 전부 삭제합니다. 되돌릴 수 없습니다.
destroy 전에 남길 값은 미리 실습 PC 로 옮겨 둡니다.